Data Mining Techniques Assignment

Nicholas Boidi[1], Pablos Tselioudis Garmendia[1], and Filippo Donghi[1]

Vrije Universiteit Amsterdam, Amsterdam, Netherlands
Group Number: 65
Student Numbers: 2892127, 2891511, 2888236

Keywords: Recommender Systems · Learning to Rank · Expedia · NDCG · Fairness

1 Introduction

This report describes our solution for Assignment 2 of Data Mining Techniques. The task is to rank hotel properties per search query (`srch_id`) so that booked hotels appear at the top, followed by clicked hotels, measured with NDCG@5 on the Kaggle competition server.

The data contains approximately five million rows for training and five million for testing. Each row is one displayed property for one search query, with rich contextual, property, and competitor information. We followed a CRISP-DM style workflow: (i) related work and business framing, (ii) exploratory analysis, (iii) feature engineering and split strategy, (iv) modeling and evaluation with at least two methods, and (v) deployment considerations including ethical AI bias analysis and mitigation.

Our best local model is a gradient boosting rank surrogate with fairness reweighting in training. It improves strongly over a recommender-style popularity baseline while reducing performance disparity between family and non-family queries.

2 Related Work and Business Understanding

The business objective is to improve ranking quality in hotel search, because better ranking increases user satisfaction and conversion (clicks/bookings). This is a recommender systems problem with context-aware ranking constraints.

Prior recommender literature highlights three relevant ideas. First, collaborative and latent-factor methods are effective at capturing user-item preference patterns [1]. Second, pairwise and listwise ranking methods (e.g., LambdaMART) are highly suitable when the evaluation target is NDCG [2]. Third, implicit-feedback recommenders (click/view/book interactions) require careful weighting of signal strength [3].

Work from the original Expedia personalized hotel search challenge also emphasizes the value of hotel characteristics, location attractiveness, user purchase history, and competitor information, and reports combining multiple ranking models for the final hotel ordering [4]. This motivated our two-model setup:

Table 1. Training/validation statistics from our reproducible run.

Statistic	Value
Training rows	958,628
Validation rows	241,372
Training queries (`srch_id`)	38,595
Validation queries (`srch_id`)	9,649
Booking rate	2.78%
Click rate	4.47%

- A recommender-style popularity model with Bayesian smoothing (strong interpretable baseline).
- A feature-rich gradient boosting model as our final method.

3 Data Understanding

3.1 Dataset Profile

We used the provided competition data only. In our reproducible training run, we loaded 1,200,000 training rows to iterate efficiently while preserving search-level structure. The resulting split statistics are shown in Table 1.

The low booking rate confirms heavy class imbalance. Since NDCG@5 is query-wise and top-heavy, calibration near the top ranks is much more important than global classification accuracy.

3.2 Exploratory Findings

We performed EDA on missingness, target sparsity, and key relationships:

- Competitor fields (`comp`) and historical visitor fields have substantial missingness.
- Price has a non-linear relationship with relevance; very low and very high prices both can underperform depending on context.
- Relative features inside a search list (e.g., price rank within `srch_id`) are more informative than absolute values alone.

Figure 1 and Figure 2 summarize these observations.

4 Data Preparation

4.1 Target Construction

Following the competition definition, we used graded relevance:

$$\text{relevance} = 5 \cdot \texttt{booking\_bool} + 1 \cdot (\texttt{click\_bool} \wedge \neg \texttt{booking\_bool}).$$

This keeps booked items as the highest-signal outcomes while still rewarding clicks.

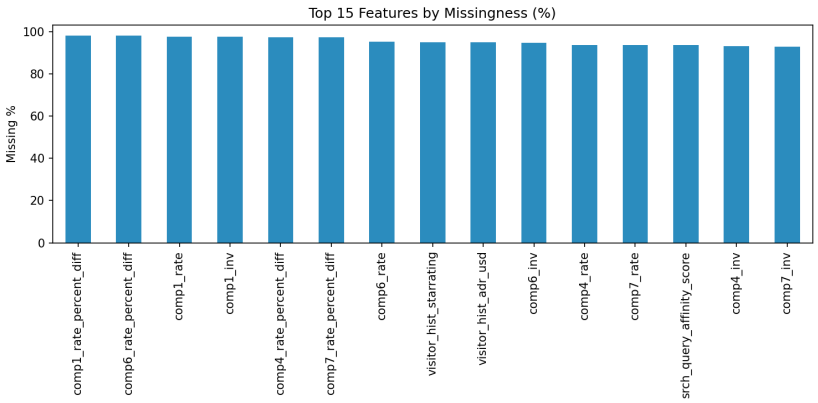

Fig. 1. Top-15 features by percentage of missing values.

4.2 Train/Validation Strategy

To avoid leakage, we split by `srch_id` (query-level split), not by row. This prevents rows from the same search appearing in both train and validation.

4.3 Feature Engineering

We engineered four groups of features:

- **Raw context/property features:** site, destination, stars, review score, price, promotion, booking window, party size, etc.
- **Competitor aggregates:** mean `comp_rate`, mean `comp_inv`, and mean `comp_rate_percent_diff` over competitors 1–8.
- **Within-query relative features:** price rank percentile, price minus search mean, stars minus search mean, review minus search mean.
- **Historical popularity priors:** global property score and destination-level score with Bayesian smoothing.

For missing values, we used model-compatible handling (tree model with native missing support) and aggregate smoothing/fallbacks where required.

5 Modeling and Evaluation

5.1 Metric

We evaluated with NDCG@5 per query and averaged over queries, matching the competition objective.

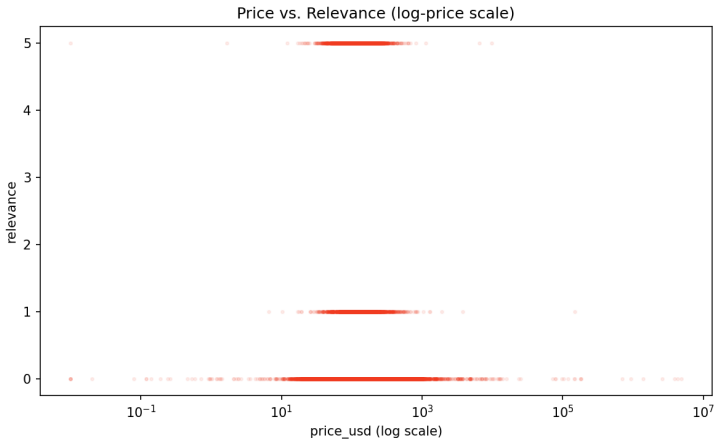

Fig. 2. Price vs. relevance on log-scale (sampled for visibility).

5.2 Technique 1: Recommender-Style Popularity Baseline

As a recommender systems technique, we ranked hotels using historical relevance priors with Bayesian smoothing:

- Global property popularity: relevance per `prop_id`.
- Destination-conditional popularity: relevance per `srch_destination_id` and `prop_id` together.
- Final score: weighted combination (0.4 global + 0.6 destination-specific).

This baseline is simple, scalable, and interpretable, and it captures collaborative signal from aggregate user interactions.

5.3 Technique 2: Gradient Boosting Rank Surrogate

Our main model is a pointwise `HistGradientBoostingRegressor` trained on graded relevance labels. Key parameters:

- `max_depth=8`
- `learning_rate=0.06`
- `max_iter=300`
- `l2_regularization=0.02`

Although pointwise, this model works well because features include query-relative comparisons and strong priors.

5.4 Results

Table 2 shows local validation performance.

The boosted model improves over the recommender baseline by about 0.0326 absolute NDCG@5, indicating that contextual and relative features provide substantial additional ranking signal.

Table 2. Validation NDCG@5 results.

Model	NDCG@5
Popularity recommender baseline	0.2487
Gradient boosting (no mitigation)	0.2813
Gradient boosting (bias-mitigated)	0.2810

5.5 What Did Not Work Well

Two observations from our iteration cycle:

- Using only absolute features (without within-query relative features) reduced top-rank quality.
- A pure global popularity ranking over-favored historically strong properties and underfit query context.

6 Deployment and Ethical AI

6.1 Scalable Deployment Discussion

A production deployment can be implemented as a two-stage architecture:

- **Offline batch layer:** refresh popularity priors and retrain gradient boosting model daily/weekly.
- **Online ranking layer:** compute lightweight query-relative features in real time, score candidate hotels, and return top-k list.
- **Monitoring:** track NDCG proxies, CTR/booking rates, latency, drift, and fairness gaps over time.

The feature design in this report is compatible with this architecture: heavy aggregates are precomputed offline, while per-query transforms are low-latency.

6.2 Bias Detection

We audited group fairness across two user-query segments:

- **Family queries:** `srch_children_count > 0`
- **Non-family queries:** `srch_children_count = 0`

Using NDCG@5 by group as the fairness metric:

- Before mitigation: family 0.2997, non-family 0.2754, gap 0.0243.

6.3 Bias Mitigation and Effectiveness

We applied a **pre-processing reweighting** technique: during model training, rows from family queries received weight 1.6 while others kept weight 1.0.

After mitigation:

- Family NDCG@5: 0.2962
- Non-family NDCG@5: 0.2761
- Gap: 0.0201

This reduced the group performance gap by about 17% with only a very small global NDCG@5 decrease (0.2813 to 0.2810), which we consider a reasonable trade-off.

7 Final Model and Submission

Our final submission model is the bias-mitigated gradient boosting rank surrogate trained with the engineered feature set and popularity priors. The generated submission file follows Kaggle format with columns `srch_id, prop_id` sorted by descending predicted relevance within each query.

8 What We Learned

This assignment improved our understanding in three ways:

- **Ranking mindset:** optimizing query-level top-k quality is different from standard classification.
- **Feature engineering impact:** relative within-query signals were more valuable than many raw features.
- **Responsible modeling:** fairness auditing and mitigation can be integrated with small performance cost.

Main challenges were data scale, missingness, and balancing relevance optimization with fairness. Unexpectedly, a simple popularity recommender already gave a strong baseline, showing the value of historical interaction priors in travel recommendation.

9 Conclusion

We developed and evaluated two ranking approaches for hotel recommendation, including a recommender-style popularity baseline and a gradient boosting model with engineered contextual features. The final model achieved the best local NDCG@5 while also reducing group disparity through pre-processing reweighting. The resulting pipeline is reproducible, scalable to production-style ranking, and aligned with both performance and ethical AI requirements of the assignment.

References

1. Koren, Y., Bell, R., Volinsky, C.: Matrix factorization techniques for recommender systems. Computer **42**(8), 30–37 (2009)
2. Burges, C.J.C.: From RankNet to LambdaRank to LambdaMART: An overview. Microsoft Research Technical Report, MSR-TR-2010-82 (2010)
3. Rendle, S., Freudenthaler, C., Gantner, Z., Schmidt-Thieme, L.: BPR: Bayesian personalized ranking from implicit feedback. In: UAI 2009, pp. 452–461 (2009)
4. Liu, X., Xu, B., Zhang, Y., Yan, Q., Pang, L., Li, Q., Sun, H., Wang, B.: Combination of diverse ranking models for personalized Expedia hotel searches. arXiv:1311.7679 (2013). `https://doi.org/10.48550/arXiv.1311.7679`
5. Jannach, D., Zanker, M., Felfernig, A., Friedrich, G.: Recommender Systems: An Introduction. Cambridge University Press, Cambridge (2010)
6. Wikipedia contributors: Discounted cumulative gain. `https://en.wikipedia.org/wiki/Discounted_cumulative_gain` (accessed 2026-04-23)